

java

 Copiar código

```
package org.example.studyplanner;

import java.time.LocalDateTime;
import java.time.LocalTime;

public class Habit implements PlannerMaterial {
    private Integer id;
    private Boolean isConcluded;
    private String name;
    private String motivation;
    private LocalTime dailyDedicationTime;
    private LocalDateTime startDate;

    // Construtores existentes
    Habit(String name, String motivation, LocalTime dailyDedicationTime, Integer id, Boolean isConcluded) {
        this.isConcluded = isConcluded;
        this.id = id;
        this.name = name;
        this.motivation = motivation;
        this.dailyDedicationTime = dailyDedicationTime;
        this.startDate = startDate;
    }

    Habit(String name, String motivation, Integer id) {
        this.isConcluded = false;
        this.id = id;
        this.name = name;
        this.motivation = motivation;
    }

    // Método estático para criar Habit de maneira mais simples
```

```

        this.motivation = motivation;
    }

    // Método estático para criar Habit de maneira mais simples
    public static Habit create(String name, String motivation, Integer dailyMinutesDedication,
                               Integer id, Integer year, Integer month, Integer day,
                               LocalTime dailyDedication = createDedicationTime(dailyHoursDedication, dailyMinutesDedication),
                               LocalDateTime startDate = createStartDate(year, month, day, hour, minute, seconds),
                               Boolean isConcluded) {
        return new Habit(name, motivation, dailyDedication, id, startDate, isConcluded);
    }

    // Método para criar LocalTime (Dedicação diária)
    private static LocalTime createDedicationTime(Integer dailyHoursDedication, Integer dailyMinutesDedication) {
        return LocalTime.of(dailyHoursDedication, dailyMinutesDedication);
    }

    // Método para criar LocalDateTime (Data de início)
    private static LocalDateTime createStartDate(Integer year, Integer month, Integer day, Integer hour, Integer minute, Integer seconds) {
        return LocalDateTime.of(year, month, day, hour, minute, seconds);
    }

    @Override
    public String toString() {
        String status = isConcluded ? "Concluded" : "Not Concluded";
        return String.format("[%d] Habit %s: %s, %s", id, name, motivation, status);
    }

    public boolean getIsConcluded() {
        return isConcluded;
    }

    public void setIsConcluded(Boolean isConcluded) {
        this.isConcluded = isConcluded;
    }

```

```
        return isConcluded;
    }

    public void setIsConcluded(Boolean isConcluded) {
        this.isConcluded = isConcluded;
    }

    public LocalDateTime getStartDate() {
        return startDate;
    }

    public Integer getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getMotivation() {
        return motivation;
    }

    public void setMotivation(String motivation) {
        this.motivation = motivation;
    }

    public LocalTime getDailyDedicationTime() {
        return dailyDedicationTime;
    }

    public void increaseDailyDedicationTime(Integer minutes) {
        this.dailyDedicationTime = dailyDedicationTime.plusMinutes(minutes);
    }
}
```

 Copiar código

 Copiar código

```
public Integer getId() {
    return id;
}

public String getName() {
    return name;
}

public String getMotivation() {
    return motivation;
}

public void setMotivation(String motivation) {
    this.motivation = motivation;
}

public LocalTime getDailyDedicationTime() {
    return dailyDedicationTime;
}

public void increaseDailyDedicationTime(Integer minutes) {
    this.dailyDedicationTime = dailyDedicationTime.plusMinutes(minutes);
}

public void decreaseDailyDedicationTime(Integer minutes) {
    this.dailyDedicationTime = dailyDedicationTime.minusMinutes(minutes);
}

public long getDaysDifference() {
    LocalDateTime now = LocalDateTime.now();
    return Duration.between(this.startDate, now).toDays();
}
```

Alterações Feitas:

1. Métodos `createDedicationTime` e `createStartDate`:

- O método `createDedicationTime` cria o objeto `LocalTime` a partir das horas e minutos de dedicação diária.
- O método `createStartDate` cria o objeto `LocalDateTime` com os parâmetros da data e hora de início.

2. Método `create`:

- Agora, o método `create` usa os novos métodos `createDedicationTime` e `createStartDate` para criar as instâncias necessárias, tornando-o mais simples e mais legível.

Benefícios da Refatoração:

1. **Redução do número de parâmetros** no método `create`, pois a criação dos objetos `LocalTime` e `LocalDateTime` é delegada para métodos especializados.
2. **Maior clareza:** A responsabilidade de criar os objetos `LocalTime` e `LocalDateTime` está bem definida e isolada em métodos próprios, facilitando a manutenção e o entendimento do código.

Com essas mudanças, o código ficou mais modular e organizado, tornando a criação de objetos `Habit` mais clara e flexível.

Se precisar de mais ajustes ou tiver dúvidas, fico à disposição! 😊

🔊 📄 👍 🔖 ↺

refatore o code smell Long Parameter List code utilizando a técnica de refatoração

```
public void assignSteps(String firstStep, String resetStudyMechanism,
String consistentStep, String seasonalSteps,
    String basicSteps, String mainObjectiveTitle, String
mainGoalTitle, String mainMaterialTopic,
    String mainTask, Integer numberOfSteps, boolean
isImportant, LocalDateTime startDate, LocalDateTime endDate) {
    DateTimeFormatter formatter =
DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");
```

```
    this.steps = new ArrayList<>(Arrays.asList(firstStep,
resetStudyMechanism, consistentStep, seasonalSteps, basicSteps,
"Number of steps: " + numberOfSteps.toString(), "Is it important to
you? " + isImportant, startDate.format(formatter),
endDate.format(formatter), mainObjectiveTitle, mainGoalTitle,
mainMaterialTopic, mainTask));
}
```

```
public void handleAssignSteps(List<String> stringProperties,
Integer numberOfSteps, boolean isImportant, LocalDateTime
startDate, LocalDateTime endDate){
    assignSteps(stringProperties.get(0), stringProperties.get(1),
stringProperties.get(2), stringProperties.get(3),
stringProperties.get(4), stringProperties.get(5),
stringProperties.get(6), stringProperties.get(7),
stringProperties.get(8), numberOfSteps, isImportant, startDate,
endDate);
}
```



```
public class StepAssignment {  
    private String firstStep;  
    private String resetStudyMechanism;  
    private String consistentStep;  
    private String seasonalSteps;  
    private String basicSteps;  
    private String mainObjectiveTitle;  
    private String mainGoalTitle;  
    private String mainMaterialTopic;  
    private String mainTask;  
    private Integer numberOfSteps;  
    private boolean isImportant;  
    private LocalDateTime startDate;  
    private LocalDateTime endDate;  
  
    // Construtor para encapsular os parâmetros  
    public StepAssignment(String firstStep, String resetStudyMechanism, String cons  
        String basicSteps, String mainObjectiveTitle, String main  
        String mainTask, Integer numberOfSteps, boolean isImporta  
  
        this.firstStep = firstStep;  
        this.resetStudyMechanism = resetStudyMechanism;  
        this.consistentStep = consistentStep;  
        this.seasonalSteps = seasonalSteps;  
        this.basicSteps = basicSteps;  
        this.mainObjectiveTitle = mainObjectiveTitle;  
        this.mainGoalTitle = mainGoalTitle;  
        this.mainMaterialTopic = mainMaterialTopic;  
        this.mainTask = mainTask;  
        this.numberOfSteps = numberOfSteps;  
        this.isImportant = isImportant;  
        this.startDate = startDate;  
        this.endDate = endDate;
```

 Copiar código



```

        this.numberOfSteps = numberOfSteps;
        this.isImportant = isImportant;
        this.startDate = startDate;
        this.endDate = endDate;
    }

    // Getter para obter uma lista formatada de informações
    public List<String> toFormattedList() {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");
        return Arrays.asList(
            firstStep, resetStudyMechanism, consistentStep, seasonalSteps, basicSteps,
            "Number of steps: " + numberOfSteps,
            "Is it important to you? " + isImportant,
            startDate.format(formatter),
            endDate.format(formatter),
            mainObjectiveTitle, mainGoalTitle, mainMaterialTopic, mainTask
        );
    }
}

```

Classe Principal:

```

java
public class StepHandler {
    private List<String> steps;

    public void assignSteps(StepAssignment stepAssignment) {
        this.steps = stepAssignment.toFormattedList();
    }

    public void handleAssignSteps(List<String> stringProperties, Integer numberOfSteps,
        LocalDateTime startDate, LocalDateTime endDate) {

```

Copiar código

 Copiar código

↓

Alterações Feitas:

Código Atualizado:

java

 Copiar código

```
package org.example.studyregistry;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Arrays;
import java.util.List;

public class StepAssignment {
    private String firstStep;
    private String resetStudyMechanism;
    private String consistentStep;
    private String seasonalSteps;
    private String basicSteps;
    private String mainObjectiveTitle;
    private String mainGoalTitle;
    private String mainMaterialTopic;
    private String mainTask;
    private Integer numberOfSteps;
    private boolean isImportant;
    private LocalDateTime startDate;
    private LocalDateTime endDate;

    // Construtor para encapsular os parâmetros
    public StepAssignment(String firstStep, String resetStudyMechanism, String consistentStep, String seasonalSteps, String basicSteps, String mainObjectiveTitle, String mainGoalTitle, String mainMaterialTopic, String mainTask, Integer numberOfSteps, boolean isImportant, LocalDateTime startDate, LocalDateTime endDate) {
        this.firstStep = firstStep;
        this.resetStudyMechanism = resetStudyMechanism;
        this.consistentStep = consistentStep;
```

```
private LocalDateTime endDate;
```

 Copiar código

```
// Construtor para encapsular os parâmetros
```

```
public StepAssignment(String firstStep, String resetStudyMechanism, String consistentStep, String basicSteps, String mainObjectiveTitle, String mainGoalTitle, String mainMaterialTopic, String mainTask, Integer numberOfSteps, boolean isImportant, LocalDateTime startDate, LocalDateTime endDate) {  
    this.firstStep = firstStep;  
    this.resetStudyMechanism = resetStudyMechanism;  
    this.consistentStep = consistentStep;  
    this.seasonalSteps = seasonalSteps;  
    this.basicSteps = basicSteps;  
    this.mainObjectiveTitle = mainObjectiveTitle;  
    this.mainGoalTitle = mainGoalTitle;  
    this.mainMaterialTopic = mainMaterialTopic;  
    this.mainTask = mainTask;  
    this.numberOfSteps = numberOfSteps;  
    this.isImportant = isImportant;  
    this.startDate = startDate;  
    this.endDate = endDate;  
}
```

```
// Getter para obter uma lista formatada de informações
```

```
public List<String> toFormattedList() {  
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");  
    return Arrays.asList(  
        firstStep,  
        resetStudyMechanism,  
        consistentStep,  
        seasonalSteps,  
        basicSteps,  
        "Number of steps: " + numberOfSteps,  
        "Is it important to you? " + isImportant,  
        "Start date: " + startDate.format(formatter),  
        "End date: " + endDate.format(formatter)  
    );  
}
```

```
        this.seasonalSteps = seasonalSteps;
        this.basicSteps = basicSteps;
        this.mainObjectiveTitle = mainObjectiveTitle;
        this.mainGoalTitle = mainGoalTitle;
        this.mainMaterialTopic = mainMaterialTopic;
        this.mainTask = mainTask;
        this.numberOfSteps = numberOfSteps;
        this.isImportant = isImportant;
        this.startDate = startDate;
        this.endDate = endDate;
    }

    // Getter para obter uma lista formatada de informações
    public List<String> toFormattedList() {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");
        return Arrays.asList(
            firstStep,
            resetStudyMechanism,
            consistentStep,
            seasonalSteps,
            basicSteps,
            "Number of steps: " + numberOfSteps,
            "Is it important to you? " + isImportant,
            (startDate != null ? startDate.format(formatter) : "Start date not set"),
            (endDate != null ? endDate.format(formatter) : "End date not set"),
            mainObjectiveTitle,
            mainGoalTitle,
            mainMaterialTopic,
            mainTask
        );
    }
}
```

Copiar código

↓

```
package org.example.studyregistry;

import java.time.LocalDateTime;

public class StepAssignment {
    private String firstStep;
    private String resetStudyMechanism;
    private String consistentStep;
    private String seasonalSteps;
    private String basicSteps;
    private String mainObjectiveTitle;
    private String mainGoalTitle;
    private String mainMaterialTopic;
    private String mainTask;
    private Integer numberOfSteps;
    private boolean isImportant;
    private LocalDateTime startDate;
    private LocalDateTime endDate;

    // Construtor privado para ser usado pelo Builder
    private StepAssignment(Builder builder) {
        this.firstStep = builder.firstStep;
        this.resetStudyMechanism = builder.resetStudyMechanism;
        this.consistentStep = builder.consistentStep;
        this.seasonalSteps = builder.seasonalSteps;
        this.basicSteps = builder.basicSteps;
        this.mainObjectiveTitle = builder.mainObjectiveTitle;
        this.mainGoalTitle = builder.mainGoalTitle;
        this.mainMaterialTopic = builder.mainMaterialTopic;
        this.mainTask = builder.mainTask;
        this.numberOfSteps = builder.numberOfSteps;
        this.isImportant = builder.isImportant;
    }
}
```

 Copiar código

```
        builder.startDate = this.startDate;  
        this.endDate = builder.endDate;  
    }  
}
```

 Copiar código

```
// Classe estática Builder
```

```
public static class Builder {
```

```
    private String firstStep;  
    private String resetStudyMechanism;  
    private String consistentStep;  
    private String seasonalSteps;  
    private String basicSteps;  
    private String mainObjectiveTitle;  
    private String mainGoalTitle;  
    private String mainMaterialTopic;  
    private String mainTask;  
    private Integer numberOfSteps;  
    private boolean isImportant;  
    private LocalDateTime startDate;  
    private LocalDateTime endDate;
```

```
    public Builder setFirstStep(String firstStep) {  
        this.firstStep = firstStep;  
        return this;  
    }
```

```
    public Builder setResetStudyMechanism(String resetStudyMechanism) {  
        this.resetStudyMechanism = resetStudyMechanism;  
        return this;  
    }
```

```
    public Builder setConsistentStep(String consistentStep) {  
        this.consistentStep = consistentStep;  
        return this;  
    }
```

```
public Builder setResetStudyMechanism(String resetStudyMechanism) {
    this.resetStudyMechanism = resetStudyMechanism;
    return this;
}

public Builder setConsistentStep(String consistentStep) {
    this.consistentStep = consistentStep;
    return this;
}

public Builder setSeasonalSteps(String seasonalSteps) {
    this.seasonalSteps = seasonalSteps;
    return this;
}

public Builder setBasicSteps(String basicSteps) {
    this.basicSteps = basicSteps;
    return this;
}

public Builder setMainObjectiveTitle(String mainObjectiveTitle) {
    this.mainObjectiveTitle = mainObjectiveTitle;
    return this;
}

public Builder setMainGoalTitle(String mainGoalTitle) {
    this.mainGoalTitle = mainGoalTitle;
    return this;
}

public Builder setMainMaterialTopic(String mainMaterialTopic) {
    this.mainMaterialTopic = mainMaterialTopic;
}
```

```
}

public Builder setMainGoalTitle(String mainGoalTitle) {
    this.mainGoalTitle = mainGoalTitle;
    return this;
}

public Builder setMainMaterialTopic(String mainMaterialTopic) {
    this.mainMaterialTopic = mainMaterialTopic;
    return this;
}

public Builder setMainTask(String mainTask) {
    this.mainTask = mainTask;
    return this;
}

public Builder setNumberOfSteps(Integer numberOfSteps) {
    this.numberOfSteps = numberOfSteps;
    return this;
}

public Builder setIsImportant(boolean isImportant) {
    this.isImportant = isImportant;
    return this;
}

public Builder setStartDate(LocalDateTime startDate) {
    this.startDate = startDate;
    return this;
}

public Builder setEndDate(LocalDateTime endDate) {
```

 Copiar código



```

        return this;
    }

    public Builder setIsImportant(boolean isImportant) {
        this.isImportant = isImportant;
        return this;
    }

    public Builder setStartDate(LocalDateTime startDate) {
        this.startDate = startDate;
        return this;
    }

    public Builder setEndDate(LocalDateTime endDate) {
        this.endDate = endDate;
        return this;
    }

    public StepAssignment build() {
        return new StepAssignment(this);
    }
}

```

 Copiar código

Exemplo de Uso do Builder

```

java

package org.example.studyregistry;

import java.time.LocalDateTime;

public class Main {

```

 Copiar código



Exemplo de Uso do Builder

java

 Copiar código

```
package org.example.studyregistry;

import java.time.LocalDateTime;

public class Main {
    public static void main(String[] args) {
        StepAssignment stepAssignment = new StepAssignment.Builder()
            .setFirstStep("Understand requirements")
            .setResetStudyMechanism("Refresh study mechanism every month")
            .setConsistentStep("Daily practice")
            .setSeasonalSteps("Adjust to semester")
            .setBasicSteps("Set up basics")
            .setMainObjectiveTitle("Learn Java")
            .setMainGoalTitle("Become proficient in Java")
            .setMainMaterialTopic("Core concepts of OOP")
            .setMainTask("Complete 5 projects")
            .setNumberOfSteps(5)
            .setIsImportant(true)
            .setStartDate(LocalDateTime.now())
            .setEndDate(LocalDateTime.now().plusMonths(3))
            .build();

        System.out.println("Step Assignment created successfully!");
    }
}
```

Benefícios da Refatoração



```
public class StepHandler {  
    private List<String> steps;  
  
    public void assignSteps(StepAssignment stepAssignment) {  
        this.steps = stepAssignment.toFormattedList();  
    }  
  
    public void handleAssignSteps(List<String> stringProperties, Integer numberOfSt  
        LocalDateTime startDate, LocalDateTime endDate) {  
        if (stringProperties.size() < 9) {  
            throw new IllegalArgumentException("The list must contain at least 9 st  
        }  
  
        // Uso do Builder para criar StepAssignment  
        StepAssignment stepAssignment = new StepAssignment.Builder()  
            .setFirstStep(stringProperties.get(0))  
            .setResetStudyMechanism(stringProperties.get(1))  
            .setConsistentStep(stringProperties.get(2))  
            .setSeasonalSteps(stringProperties.get(3))  
            .setBasicSteps(stringProperties.get(4))  
            .setMainObjectiveTitle(stringProperties.get(5))  
            .setMainGoalTitle(stringProperties.get(6))  
            .setMainMaterialTopic(stringProperties.get(7))  
            .setMainTask(stringProperties.get(8))  
            .setNumberOfSteps(numberOfSteps)  
            .setIsImportant(isImportant)  
            .setStartDate(startDate)  
            .setEndDate(endDate)  
            .build();  
  
        assignSteps(stepAssignment);  
    }  
}
```

 Copiar código

Classe `StudyPlan` com `StepHandler` Atualizado:

java

 Copiar código

```
package org.example.studyregistry;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class StudyPlan extends Registry {
    private StudyObjective objective;
    private List<String> steps;

    public StudyPlan(String planName, StudyObjective objective, List<StudyMaterial> materials) {
        this.name = planName;
        this.objective = objective;
        this.steps = new ArrayList<>();
    }

    @Override
    public String toString() {
        return "Plan: " + name + ",\nObjective: " + objective.getDescription() + ",\nSteps: " + steps;
    }

    public List<String> getSteps() {
        return steps;
    }

    public StudyObjective getObjective() {
        return objective;
    }
}
```

```

        steps.add(toAdd);
    }

    public static class StepHandler {
        private List<String> steps;

        public void assignSteps(StepAssignment stepAssignment) {
            this.steps = stepAssignment.toFormattedList();
        }

        public void handleAssignSteps(List<String> stringProperties, Integer numberOfSteps,
                                       LocalDateTime startDate, LocalDateTime endDate) {
            if (stringProperties.size() < 9) {
                throw new IllegalArgumentException("The list must contain at least 9 elements");
            }

            // Uso do Builder para criar StepAssignment
            StepAssignment stepAssignment = new StepAssignment.Builder()
                .setFirstStep(stringProperties.get(0))
                .setResetStudyMechanism(stringProperties.get(1))
                .setConsistentStep(stringProperties.get(2))
                .setSeasonalSteps(stringProperties.get(3))
                .setBasicSteps(stringProperties.get(4))
                .setMainObjectiveTitle(stringProperties.get(5))
                .setMainGoalTitle(stringProperties.get(6))
                .setMainMaterialTopic(stringProperties.get(7))
                .setMainTask(stringProperties.get(8))
                .setNumberOfSteps(numberOfSteps)
                .setIsImportant(isImportant)
                .setStartDate(startDate)
                .setEndDate(endDate)
            };
        }
    }

```

```
        .setEndDate(endDate)
        .build();

    assignSteps(stepAssignment);
    }
}
}
```

Alterações Realizadas:

1. Uso do `Builder` no `StepHandler`:

- Substituiu o construtor longo de `StepAssignment` pelo `Builder`.

2. Manutenção de Flexibilidade:

- O código agora está mais organizado e evita a confusão de parâmetros posicionais.

3. Melhor Manutenção:

- Alterações no construtor de `StepAssignment` ou nos seus campos exigem apenas atualizações no `Builder`, sem necessidade de alterar múltiplos pontos de uso no código.

Benefícios:

- **Legibilidade:** O uso do `Builder` torna o código mais intuitivo e fácil de entender.
- **Flexibilidade:** Adicionar ou remover campos no futuro é mais simples.
- **Menos Erros:** Atribuições erradas de parâmetros devido a ordem incorreta são evitadas.

🔊 📄 👍 🗨️ ↻



```
public void assignSteps(String firstStep, String resetStudyMechanism,  
String consistentStep, String seasonalSteps,
```

```
String basicSteps, String mainObjectiveTitle, String mainGoalTitle,  
String mainMaterialTopic,
```

```
String mainTask, Integer numberOfSteps, boolean isImportant,  
LocalDateTime startDate, LocalDateTime endDate) {
```

```
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-  
MM-yyyy HH:mm:ss");
```

```
this.steps = new ArrayList<> (Arrays.asList(firstStep,  
resetStudyMechanism, consistentStep, seasonalSteps, basicSteps,  
"Number of steps: " + numberOfSteps.toString(), "Is it important to  
you? " + isImportant, startDate.format(formatter),  
endDate.format(formatter), mainObjectiveTitle, mainGoalTitle,  
mainMaterialTopic, mainTask));
```

```
}
```

```
public void handleAssignSteps(List<String> stringProperties, Integer  
numberOfSteps, boolean isImportant, LocalDateTime startDate,  
LocalDateTime endDate){
```

```
assignSteps(stringProperties.get(0), stringProperties.get(1),  
stringProperties.get(2), stringProperties.get(3),
```

java

 Copiar código

```
public class StepDetails {  
    private List<String> stringProperties;  
    private Integer numberOfSteps;  
    private boolean isImportant;  
    private LocalDateTime startDate;  
    private LocalDateTime endDate;  
  
    public StepDetails(List<String> stringProperties, Integer numberOfSteps, boolean isImportant, LocalDateTime startDate, LocalDateTime endDate) {  
        this.stringProperties = stringProperties;  
        this.numberOfSteps = numberOfSteps;  
        this.isImportant = isImportant;  
        this.startDate = startDate;  
        this.endDate = endDate;  
    }  
  
    // Getters e setters  
    public List<String> getStringProperties() {  
        return stringProperties;  
    }  
  
    public Integer getNumberOfSteps() {  
        return numberOfSteps;  
    }  
  
    public boolean isImportant() {  
        return isImportant;  
    }  
  
    public LocalDateTime getStartDate() {  
        return startDate;  
    }  
}
```

```

    public StepDetails(List<String> stringProperties, Integer numberOfSteps, boolean isImportant, LocalDateTime startDate, LocalDateTime endDate) {
        this.stringProperties = stringProperties;
        this.numberOfSteps = numberOfSteps;
        this.isImportant = isImportant;
        this.startDate = startDate;
        this.endDate = endDate;
    }

    // Getters e setters
    public List<String> getStringProperties() {
        return stringProperties;
    }

    public Integer getNumberOfSteps() {
        return numberOfSteps;
    }

    public boolean isImportant() {
        return isImportant;
    }

    public LocalDateTime getStartDate() {
        return startDate;
    }

    public LocalDateTime getEndDate() {
        return endDate;
    }
}

```



2. Alterar os métodos para usar o novo objeto:

```
}
```

2. Alterar os métodos para usar o novo objeto:

java

 Copiar código

```
public void assignSteps(StepDetails stepDetails) {
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss")

    this.steps = new ArrayList<>(Arrays.asList(
        stepDetails.getStringProperties().get(0),
        stepDetails.getStringProperties().get(1),
        stepDetails.getStringProperties().get(2),
        stepDetails.getStringProperties().get(3),
        stepDetails.getStringProperties().get(4),
        stepDetails.getStringProperties().get(5),
        stepDetails.getStringProperties().get(6),
        "Number of steps: " + stepDetails.getNumberOfSteps().toString(),
        "Is it important to you? " + stepDetails.isImportant(),
        stepDetails.getStartDate().format(formatter),
        stepDetails.getEndDate().format(formatter)
    ));
}

public void handleAssignSteps(StepDetails stepDetails) {
    assignSteps(stepDetails);
}
```

3. Uso do método atualizado:



```

        if (stepDetails.isImportant()) {
            stepDetails.getStartDate().format(formatter),
            stepDetails.getEndDate().format(formatter)
        });
    }

    public void handleAssignSteps(StepDetails stepDetails) {
        assignSteps(stepDetails);
    }

```

3. Uso do método atualizado:

java

 Copiar código

```

List<String> stringProperties = Arrays.asList("First step", "Reset study mechanism");
StepDetails stepDetails = new StepDetails(stringProperties, 5, true, LocalDateTime.now());

handleAssignSteps(stepDetails);

```

Vantagens do "Introduce Parameter Object"

- **Legibilidade:** Reduz a complexidade da assinatura do método.
- **Manutenção:** Modificar ou adicionar novos parâmetros é mais fácil.
- **Reutilização:** Dados encapsulados no objeto podem ser usados em outros lugares do sistema.

Outras técnicas, como **Extract Method** ou **Extract Class**, podem ser úteis dependendo do contexto, mas para resolver diretamente a lista longa de parâmetros, o **Introduce Parameter Object** é a técnica mais adequada.

como fica isso depois da refatoração?

```
private void handleSetSteps(StudyPlan studyPlan){
    handleMethodHeader("(Study Plan Edit)");
    System.out.println("Type the following info: String firstStep,
String resetStudyMechanism, String consistentStep, " +
        "String seasonalSteps, String basicSteps, String
mainObjectiveTitle, String mainGoalTitle, String mainMaterialTopic, "
+
        "String mainTask, @NotNull Integer numberOfSteps,
boolean isImportant. " +
        "The Date to start is today, the date to end is x days from
now, type the quantity of days\n");
    LocalDateTime createdAT = LocalDateTime.now();
    studyPlan.assignSteps(getInput(), getInput(), getInput(),
getInput(), getInput(), getInput(), getInput(), getInput(), getInput(),
Integer.parseInt(getInput()),
Boolean.parseBoolean(getInput()), createdAT,
createdAT.plusDays(Long.parseLong(getInput())));
}
```



Após a refatoração para resolver o problema de **Long Parameter List**, o método pode ser simplificado utilizando a classe `StepDetails` (ou um objeto similar para encapsular os dados). A refatoração torna o código mais legível e fácil de manter. Veja como ficaria o método refatorado:



```

private void handleSetSteps(StudyPlan studyPlan) {
    handleMethodHeader("(Study Plan Edit)");
    System.out.println("Type the following info: \n" +
        "- String firstStep\n" +
        "- String resetStudyMechanism\n" +
        "- String consistentStep\n" +
        "- String seasonalSteps\n" +
        "- String basicSteps\n" +
        "- String mainObjectiveTitle\n" +
        "- String mainGoalTitle\n" +
        "- String mainMaterialTopic\n" +
        "- String mainTask\n" +
        "- Integer numberOfSteps\n" +
        "- boolean isImportant\n" +
        "- Quantity of days from today for the end date\n");

    // Coleta os dados de entrada
    List<String> stringProperties = new ArrayList<>();
    for (int i = 0; i < 9; i++) { // 9 Strings de entrada
        stringProperties.add(getInput());
    }
    Integer numberOfSteps = Integer.parseInt(getInput());
    boolean isImportant = Boolean.parseBoolean(getInput());
    LocalDateTime startDate = LocalDateTime.now();
    LocalDateTime endDate = startDate.plusDays(Long.parseLong(getInput()));

    // Cria o objeto StepDetails
    StepDetails stepDetails = new StepDetails(stringProperties, numberOfSteps, isIn

    // Passa o objeto para o método assignSteps
    studyPlan.assignSteps(stepDetail
}

```

java

 Copiar código

```
public class StepDetails {
    private List<String> stringProperties;
    private Integer numberOfSteps;
    private boolean isImportant;
    private LocalDateTime startDate;
    private LocalDateTime endDate;

    public StepDetails(List<String> stringProperties, Integer numberOfSteps, boolean isImportant, LocalDateTime startDate, LocalDateTime endDate) {
        this.stringProperties = stringProperties;
        this.numberOfSteps = numberOfSteps;
        this.isImportant = isImportant;
        this.startDate = startDate;
        this.endDate = endDate;
    }

    // Getters e setters
    public List<String> getStringProperties() {
        return stringProperties;
    }

    public Integer getNumberOfSteps() {
        return numberOfSteps;
    }

    public boolean isImportant() {
        return isImportant;
    }

    public LocalDateTime getStartDate() {
        return startDate;
    }
}
```

```
        this.endDate = endDate;
    }

    // Getters e setters
    public List<String> getStringProperties() {
        return stringProperties;
    }

    public Integer getNumberOfSteps() {
        return numberOfSteps;
    }

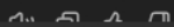
    public boolean isImportant() {
        return isImportant;
    }

    public LocalDateTime getStartDate() {
        return startDate;
    }

    public LocalDateTime getEndDate() {
        return endDate;
    }
}
```

[Copiar código](#)

Essa abordagem elimina o excesso de parâmetros na chamada do método, centraliza a lógica em um objeto dedicado e melhora a clareza ger.  código.



Essa abordagem elimina o excesso de parâmetros na chamada do método, centraliza a lógica em um objeto dedicado e melhora a clareza geral do código.

```
void assignSteps(){
    List<String> stringProperties = List.of("firstStep",
    "resetStudyMechanism", "consistentStep", "seasonalSteps",
    "basicSteps", "mainObjectiveTitle", "mainGoalTitle",
    "mainMaterialTopic", "mainTask");
    Integer numberOfSteps = 20;
    boolean isImportant = true;
    startDateTest = LocalDateTime.now();
    LocalDateTime startDate = startDateTest;
    LocalDateTime endDate = startDate.plusDays(10);
    studyPlan.handleAssignSteps(stringProperties, numberOfSteps,
    isImportant, startDate, endDate);
}
```

como fica isso depois da refatoração?



Após a refatoração para utilizar a classe `StepDetails` e resolver o problema de **Long Parameter List**, o método `assignSteps` pode ser reescrito para criar e passar diretamente um objeto encapsulando os parâmetros. Veja como ficaria:

Método refatorado:



Método refatorado:

java

 Copiar código

```
void assignSteps() {
    List<String> stringProperties = List.of(
        "firstStep", "resetStudyMechanism", "consistentStep", "seasonalSteps",
        "basicSteps", "mainObjectiveTitle", "mainGoalTitle", "mainMaterialTopic", "
    );
    Integer numberOfSteps = 20;
    boolean isImportant = true;
    startDateTest = LocalDateTime.now();
    LocalDateTime startDate = startDateTest;
    LocalDateTime endDate = startDate.plusDays(10);

    // Criação do objeto StepDetails
    StepDetails stepDetails = new StepDetails(stringProperties, numberOfSteps, isIm

    // Passa o objeto diretamente ao método
    studyPlan.handleAssignSteps(stepDetails);
}
```

O que mudou?

1. Criação do objeto `StepDetails`:

- Agrupa os dados necessários (propriedades de string, número de etapas, importância, e datas) em um único objeto

```
public void setUpWeek(String planName, String objectiveTitle, String
objectiveDescription, String materialTopic,
    String materialFormat, String goal, String reminderTitle,
String reminderDescription,
    String mainTaskTitle, String mainHabit, String
mainCardStudy){
    this.weekResponsibilities = new ArrayList<>();
    this.weekResponsibilities.addAll(Arrays.asList(planName,
objectiveTitle, objectiveDescription, materialTopic, materialFormat,
goal, reminderTitle, reminderDescription, mainTaskTitle, mainHabit,
mainCardStudy));
}
```

```
public void handleSetUpWeek(List<String> stringProperties){
    setUpWeek(stringProperties.get(0), stringProperties.get(1),
stringProperties.get(2), stringProperties.get(3),
    stringProperties.get(4), stringProperties.get(5),
stringProperties.get(6), stringProperties.get(7),
    stringProperties.get(8), stringProperties.get(9),
stringProperties.get(10));
}
```

qual o melhor pra resolver esse problema de Long Parameter List

Extract Method

Move Field



Move Method

```
public class WeekSetup {  
    private String planName;  
    private String objectiveTitle;  
    private String objectiveDescription;  
    private String materialTopic;  
    private String materialFormat;  
    private String goal;  
    private String reminderTitle;  
    private String reminderDescription;  
    private String mainTaskTitle;  
    private String mainHabit;  
    private String mainCardStudy;  
  
    // Construtor e getters/setters  
    public WeekSetup(String planName, String objectiveTitle, String objectiveDescription,  
        String materialFormat, String goal, String reminderTitle, String mainTaskTitle,  
        String mainHabit, String mainCardStudy)  
    {  
        this.planName = planName;  
        this.objectiveTitle = objectiveTitle;  
        this.objectiveDescription = objectiveDescription;  
        this.materialTopic = materialTopic;  
        this.materialFormat = materialFormat;  
        this.goal = goal;  
        this.reminderTitle = reminderTitle;  
        this.reminderDescription = reminderDescription;  
        this.mainTaskTitle = mainTaskTitle;  
        this.mainHabit = mainHabit;  
        this.mainCardStudy = mainCardStudy;  
    }  
  
    // Getters e setters podem ser reutilizados aqui  
}
```

 Copiar código



 Copiar código

```
// Getters e setters podem ser gerados aqui  
}
```

Refatorando `setUpWeek` :

java

 Copiar código

```
public void setUpWeek(WeekSetup weekSetup) {  
    this.weekResponsibilities = new ArrayList<>();  
    this.weekResponsibilities.addAll(Arrays.asList(  
        weekSetup.getPlanName(),  
        weekSetup.getObjectiveTitle(),  
        weekSetup.getObjectiveDescription(),  
        weekSetup.getMaterialTopic(),  
        weekSetup.getMaterialFormat(),  
        weekSetup.getGoal(),  
        weekSetup.getReminderTitle(),  
        weekSetup.getReminderDescription(),  
        weekSetup.getMainTaskTitle(),  
        weekSetup.getMainHabit(),  
        weekSetup.getMainCardStudy()  
    ));  
}  
  
public void handleSetUpWeek(WeekSetup weekSetup) {  
    setUpWeek(weekSetup);  
}
```

Benefícios:



1. **Redução da complexidade:** Em vez de lidar com uma longa lista de parâmetros, você trabalha

Agora a classe possui todos os getters e setters necessários para acessar e modificar os atributos.



como fica isso apos a refatoração?

```
private void getWeekInfo(){
    System.out.println("(Study Task Manager Week Set Up) Type the
    following info: String planName, String objectiveTitle, " +
        "String objectiveDescription, String materialTopic, String
    materialFormat, String goal, String reminderTitle, " +
        "String reminderDescription, String mainTaskTitle, String
    mainHabit, String mainCardStudy");
    studyTaskManager.setUpWeek(getInput(), getInput(), getInput(),
    getInput(), getInput(), getInput(), getInput(), getInput(),
    getInput(), getInput(), getInput());
}
```



Após a refatoração para usar a classe `WeekSetup`, o método `getWeekInfo` pode ser simplificado para criar um objeto `WeekSetup` e passá-lo para o método `setUpWeek`. O código refatorado ficaria assim:

Código Refatorado:

java

 Copiar código

```
private void getWeekInfo() {
    System.out.println("(Study Task Manager Week Set Up) Type the following info: p
        \"objectiveDescription, materialTopic, materialFormat, goal, reminderTit
        \"mainTaskTitle, mainHabit, mainCardStudy\");
}
```

Código Refatorado:

java

 Copiar código

```
private void getWeekInfo() {
    System.out.println("(Study Task Manager Week Set Up) Type the following info: p
        \"objectiveDescription, materialTopic, materialFormat, goal, reminderTit
        \"mainTaskTitle, mainHabit, mainCardStudy\");

    // Captura os inputs do usuário e cria um objeto WeekSetup
    WeekSetup weekSetup = new WeekSetup(
        getInput(), // planName
        getInput(), // objectiveTitle
        getInput(), // objectiveDescription
        getInput(), // materialTopic
        getInput(), // materialFormat
        getInput(), // goal
        getInput(), // reminderTitle
        getInput(), // reminderDescription
        getInput(), // mainTaskTitle
        getInput(), // mainHabit
        getInput() // mainCardStudy
    );

    // Passa o objeto WeekSetup para o método setUpWeek
    studyTaskManager.setUpWeek(weekSetup);
}
```

Mudanças realizadas:

1. Criação de um objeto `WeekSetup`:



Os inputs do usuário são capturados e usados diretamente para instanciar um objeto da classe

simples e terá menor impacto no restante do código.

- **Consistência:** O uso de um objeto encapsula os dados relacionados e evita potenciais erros causados pela ordem dos parâmetros.



como fica isso apos a refatoração

```
static void addWeekResponsibilities(){
    List<String> stringProperties = List.of("planName",
        "objectiveTitle", "objectiveDescription", "materialTopic",
        "materialFormat", "goal", "reminderTitle",
        "reminderDescription",
        "mainTaskTitle", "mainHabit", "mainCardStudy");
    studyTaskManager.handleSetUpWeek(stringProperties);
}
```



Após a refatoração, onde a lista de parâmetros é substituída por um objeto `WeekSetup`, o método `addWeekResponsibilities` pode ser simplificado para criar um objeto `WeekSetup` diretamente e passar esse objeto para o método `handleSetUpWeek`. Aqui está como ele ficaria:

Código Refatorado:

java

Copiar código

```
static void addWeekResponsibilities() {
    // Cria um objeto WeekSetup usando os valores da lista de strings
    WeekSetup weekSetup = new WeekSetup(
        "planName",           // planName
        "objectiveTitle",     // objectiveTitle
        "objectiveDescription", // objectiveDescription
    );
    studyTaskManager.handleSetUpWeek(weekSetup);
}
```

`addWeekResponsibilities` pode ser simplificado para criar um objeto `WeekSetup` diretamente e passar esse objeto para o método `handleSetUpWeek`. Aqui está como ele ficaria:

Código Refatorado:

java

 Copiar código

```
static void addWeekResponsibilities() {  
    // Cria um objeto WeekSetup usando os valores da lista de strings  
    WeekSetup weekSetup = new WeekSetup(  
        "planName",          // planName  
        "objectiveTitle",    // objectiveTitle  
        "objectiveDescription", // objectiveDescription  
        "materialTopic",     // materialTopic  
        "materialFormat",    // materialFormat  
        "goal",              // goal  
        "reminderTitle",     // reminderTitle  
        "reminderDescription", // reminderDescription  
        "mainTaskTitle",     // mainTaskTitle  
        "mainHabit",         // mainHabit  
        "mainCardStudy"      // mainCardStudy  
    );  
  
    // Passa o objeto WeekSetup para o método handleSetUpWeek  
    studyTaskManager.handleSetUpWeek(weekSetup);  
}
```

Mudanças:

1. Substituição de `List<String>` por `WeekSetup`:

Em vez de criar uma lista de strings e passá-la para o método, agora você cria um objeto

```
public void setUpWeek(String planName, String objectiveTitle, String
objectiveDescription, String materialTopic,
                    String materialFormat, String goal, String reminderTitle,
String reminderDescription,
                    String mainTaskTitle, String mainHabit, String
mainCardStudy){
    this.weekResponsibilities = new ArrayList<>();
    this.weekResponsibilities.addAll(Arrays.asList(planName,
objectiveTitle, objectiveDescription, materialTopic, materialFormat,
goal, reminderTitle, reminderDescription, mainTaskTitle, mainHabit,
mainCardStudy));
}
```

qual o melhor pra resolver esse problema de Long Parameter List

- Extract Method
- Move Field
- Move Method
- Extract Class
- Extract Subclass
- Decompose Conditional
- Extract Superclass
- Replace Inheritance with Delegation
- Introduce Parameter Object
- Replace Temp with Query
- Encapsulate Field



Para resolver o problema de **Long Parameter List**, a abordagem mais adequada seria:



Introduce Parameter Object

```
public class WeekSetup {  
    private String planName;  
    private String objectiveTitle;  
    private String objectiveDescription;  
    private String materialTopic;  
    private String materialFormat;  
    private String goal;  
    private String reminderTitle;  
    private String reminderDescription;  
    private String mainTaskTitle;  
    private String mainHabit;  
    private String mainCardStudy;  
  
    // Construtor  
    public WeekSetup(String planName, String objectiveTitle, String objectiveDe  
        String materialFormat, String goal, String reminderTitle,  
        String mainTaskTitle, String mainHabit, String mainCardStu  
        this.planName = planName;  
        this.objectiveTitle = objectiveTitle;  
        this.objectiveDescription = objectiveDescription;  
        this.materialTopic = materialTopic;  
        this.materialFormat = materialFormat;  
        this.goal = goal;  
        this.reminderTitle = reminderTitle;  
        this.reminderDescription = reminderDescription;  
        this.mainTaskTitle = mainTaskTitle;  
        this.mainHabit = mainHabit;  
        this.mainCardStudy = mainCardStudy;  
    }  
  
    // Getters e setters (opcional) ↓  
}
```

 Copiar código

```

        this.reminderDescription = reminderDescription;
        this.mainTaskTitle = mainTaskTitle;
        this.mainHabit = mainHabit;
        this.mainCardStudy = mainCardStudy;
    }

    // Getters e setters (opcional)
}

```

 Copiar código

2. Modificar o método para usar o objeto de parâmetro:

```

java

public void setUpWeek(WeekSetup weekSetup) {
    this.weekResponsibilities = new ArrayList<>();
    this.weekResponsibilities.addAll(Arrays.asList(
        weekSetup.getPlanName(),
        weekSetup.getObjectiveTitle(),
        weekSetup.getObjectiveDescription(),
        weekSetup.getMaterialTopic(),
        weekSetup.getMaterialFormat(),
        weekSetup.getGoal(),
        weekSetup.getReminderTitle(),
        weekSetup.getReminderDescription(),
        weekSetup.getMainTaskTitle(),
        weekSetup.getMainHabit(),
        weekSetup.getMainCardStudy()
    ));
}

```

 Copiar código

Essa abordagem melhora a legibilidade, simplifica a assinatura do método e facilita a manutenção

```
}
```

[Copiar código](#)

Agora a classe `WeekSetup` possui todos os getters e setters necessários para acessar e modificar os atributos.



refatore isso

```
public void handleSetUpWeek(List<String> stringProperties){
    setUpWeek(stringProperties.get(0), stringProperties.get(1),
stringProperties.get(2), stringProperties.get(3),
        stringProperties.get(4), stringProperties.get(5),
stringProperties.get(6), stringProperties.get(7),
        stringProperties.get(8), stringProperties.get(9),
stringProperties.get(10));
}
```



Para refatorar o método `handleSetUpWeek` e melhorar a legibilidade e manutenção, você pode usar a classe `WeekSetup` que criamos anteriormente. Em vez de passar uma lista de strings e acessar cada elemento pelo índice, você pode instanciar um objeto `WeekSetup` e passá-lo diretamente para o método `setUpWeek`.

Aqui está a refatoração:

```
java
```

[Copiar código](#)

```
public void handleSetUpWeek(List<String> stringProperties) {
    if (stringProperties == null || stringProperties.size() < 11) {
        throw new IllegalArgumentException("Insufficient properties to set up the w
    }
}
```



Aqui está a refatoração:

java

 Copiar código

```
public void handleSetUpWeek(List<String> stringProperties) {
    if (stringProperties == null || stringProperties.size() < 11) {
        throw new IllegalArgumentException("Insufficient properties to set up the w
    }

    WeekSetup weekSetup = new WeekSetup(
        stringProperties.get(0), // planName
        stringProperties.get(1), // objectiveTitle
        stringProperties.get(2), // objectiveDescription
        stringProperties.get(3), // materialTopic
        stringProperties.get(4), // materialFormat
        stringProperties.get(5), // goal
        stringProperties.get(6), // reminderTitle
        stringProperties.get(7), // reminderDescription
        stringProperties.get(8), // mainTaskTitle
        stringProperties.get(9), // mainHabit
        stringProperties.get(10) // mainCardStudy
    );

    setUpWeek(weekSetup);
}
```

Alterações feitas:

1. **Validação:** Adicionei uma verificação para garantir que a lista `stringProperties` não seja `null` e tenha pelo menos 11 elementos.
2. **Uso do `WeekSetup`:** Criei uma instância de `WeekSetup` passando os elementos da lista.

simplificando a chamada.

Essa abordagem melhora a clareza e reduz a probabilidade de erros relacionados a índices de lista.

refatore isso de acordo

```
private void getWeekInfo(){
    System.out.println("(Study Task Manager Week Set Up) Type the
    following info: String planName, String objectiveTitle, " +
        "String objectiveDescription, String materialTopic, String
    materialFormat, String goal, String reminderTitle, " +
        "String reminderDescription, String mainTaskTitle, String
    mainHabit, String mainCardStudy");
    studyTaskManager.setUpWeek(getInput(), getInput(), getInput(),
    getInput(), getInput(), getInput(), getInput(), getInput(),
    getInput(), getInput(), getInput());
}
```



Para refatorar o método `getWeekInfo`, vamos utilizar a classe `WeekSetup` e reduzir a repetição de chamadas para `getInput()`. Podemos armazenar as entradas do usuário em uma lista e depois usá-las para criar uma instância de `WeekSetup`.

Aqui está a refatoração:

java

 Copiar código

```
private void getWeekInfo() {
    System.out.println("(Study Task Manager Week Set Up) Type the following info: "
        "planName, objectiveTitle, objectiveDescription, materialTopic, materialFor
        reminderTitle, reminderDescription, mainTaskTitle, mainHabit, mainCardStuc
```

usados para criar uma instância de `WeekSetup`.

Aqui está a refatoração:

java

 Copiar código

```
private void getWeekInfo() {
    System.out.println("(Study Task Manager Week Set Up) Type the following info: "
        + "planName, objectiveTitle, objectiveDescription, materialTopic, materialFormat, goal, "
        + "reminderTitle, reminderDescription, mainTaskTitle, mainHabit, mainCardStudy");

    List<String> inputs = new ArrayList<>();
    for (int i = 0; i < 11; i++) {
        inputs.add(getInput());
    }

    WeekSetup weekSetup = new WeekSetup(
        inputs.get(0), // planName
        inputs.get(1), // objectiveTitle
        inputs.get(2), // objectiveDescription
        inputs.get(3), // materialTopic
        inputs.get(4), // materialFormat
        inputs.get(5), // goal
        inputs.get(6), // reminderTitle
        inputs.get(7), // reminderDescription
        inputs.get(8), // mainTaskTitle
        inputs.get(9), // mainHabit
        inputs.get(10) // mainCardStudy
    );

    studyTaskManager.setUpWeek(weekSetup);
}
```

